Data Handling and Infrastructure for AI

HPC

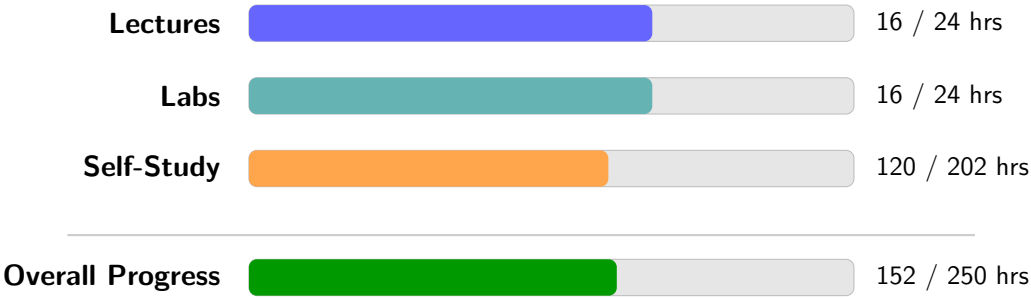
Tamás Grósz

SETU

2026



Workload


Lectures
16 / 24 hrs
Labs
16 / 24 hrs
Self-Study
120 / 202 hrs
Overall Progress
152 / 250 hrs

Learning Objectives

By the end of this lecture, you should be able to:

- Explain what High Performance Computing (HPC) is.
- Distinguish between parallelism and distributed computing.
- Explain CPU, GPU and accelerator-based computation.
- Understand the main bottlenecks in HPC workloads.
- Explain shared-memory and distributed-memory systems.
- Describe the basics of MPI and collective communication.
- Connect HPC to distributed ML training.
- Understand how HPC workloads are scheduled and managed.
- Relate HPC to Docker, Kubernetes and OpenShift.
- Identify when HPC is useful for an ML project.

Connection to previous Lectures

We have already discussed:

- Data types, storage and data pipelines
- Batch processing and streaming
- ML models, loss functions and SGD
- Epochs, steps and mini-batches
- Learning rates and hyperparameter optimisation
- GPUs and TPUs
- MLflow and model versioning
- Docker, Kubernetes and OpenShift

Now we ask:

What happens when one machine is no longer enough?

What Is High Performance Computing?

High Performance Computing (HPC) uses powerful computing resources to solve computationally demanding problems.

An HPC system may contain:

- multiple CPUs,
- many CPU cores,
- GPUs or other accelerators,
- large amounts of memory,
- high-speed interconnects,
- high-performance storage.

$$\text{HPC} = \text{Compute} + \text{Memory} + \text{Communication} + \text{Storage}$$

Key idea

HPC is about making large computational workloads practical through appropriate hardware and parallel execution.

Why Do We Need HPC?

Consider a workload that takes 100 hours on a single machine.
Possible approaches:

- Use a faster processor.
- Use a GPU.
- Optimise the algorithm.
- Use more memory.
- Split the workload across multiple machines.

HPC becomes useful when a problem:

- takes too long,
- requires more memory than one machine has,
- requires more compute than one machine can provide,
- contains work that can be executed in parallel.

HPC and Machine Learning

Modern ML workloads can require enormous computation.

- Training a deep neural network
- Training a Transformer
- Hyperparameter optimisation
- Cross-validation
- Distributed inference
- Processing very large datasets

Recall the training update:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

Computing the gradient can involve billions or trillions of numerical operations.

Connection

HPC provides the computational resources and parallel methods that make large-scale ML training possible.

Three Ways to Improve Performance

There are three broad strategies:

Optimise

Improve the algorithm.

Reduce unnecessary computation.

Accelerate

Use specialised hardware.

GPUs / TPUs.

Parallelise

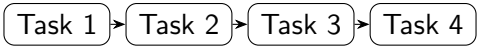
Use multiple cores or machines.

Distributed execution.

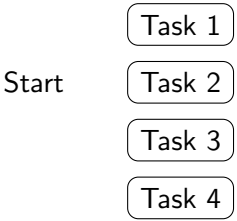
Better algorithm + Better hardware + Parallelism

Serial vs Parallel Computation

A serial program performs operations one after another.



A parallel program executes independent operations simultaneously.



Important

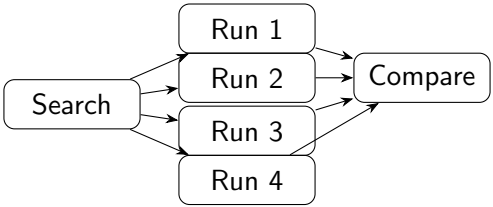
Parallelism is only useful when the workload can be divided into work that can proceed concurrently.

What Can We Parallelise?

Consider hyperparameter optimisation.
We want to evaluate:

$$\lambda_1, \lambda_2, \lambda_3, \lambda_4$$

Each configuration requires an independent training run.



This is a form of **task parallelism**.
Each experiment can run independently, provided sufficient resources are available.

Data Parallelism

Recall SGD and mini-batches:

$$B = \{(x_1, y_1), \ldots, (x_m, y_m)\}$$

The loss gradient is:

$$\nabla_\theta L \approx \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta L_i$$

The gradient contributions can be computed in parallel.

$$\text{GPU } 1 : x_1, \ldots, x_{m/2}$$

$$\text{GPU } 2 : x_{m/2+1}, \ldots, x_m$$

Then the results are combined.

Connection to previous lecture

Mini-batch SGD provides a natural opportunity for data parallelism.

The Limits of Parallelism

More processors do not always mean proportionally faster execution.

$$T_1 = 100 \text{ seconds}$$

If we use 4 processors, ideal speedup would be:

$$S(4) = 4$$

and:

$$T_4 = 25 \text{ seconds}$$

But real workloads may require:

- communication,
- synchronisation,
- data transfer,
- duplicated work,
- serial computation.

Amdahl's Law

Amdahl's Law estimates the maximum speedup from parallelisation.

$$P = \text{parallel fraction of the workload}$$

$$1 - P = \text{serial fraction}$$

With N processors:

$$\boxed{S(N) = \frac{1}{(1-P) + \frac{P}{N}}}$$

Example:

$$P = 0.9, \qquad N = 10$$

$$S(10) = \frac{1}{0.1 + \frac{0.9}{10}} = 5.26$$

Lesson

The serial portion of a program limits the benefit of parallel execution.

Strong Scaling vs Weak Scaling

Strong scaling

Keep the problem size fixed and increase the number of processors.

$$N_{\text{data}} = \text{constant}$$

Question:

How much faster can we solve the same problem?

Weak scaling

Increase the problem size as we increase the number of processors.

$$\frac{N_{\text{data}}}{N_{\text{processors}}} = \text{constant}$$

Question:

Can we solve a proportionally larger problem in similar time?

CPU Architecture

A CPU is designed for general-purpose computation.
Typical characteristics:

- relatively few powerful cores,
- complex control logic,
- large caches,
- strong performance for serial code,
- support for diverse workloads.

CPUs are useful for:

- data preprocessing,
- orchestration,
- operating system tasks,
- control flow,
- workloads with limited parallelism.

GPU Architecture

GPUs are designed to execute many similar operations in parallel.
Typical strengths:

- large numbers of compute units,
- high memory bandwidth,
- efficient matrix and tensor operations,
- high throughput for parallel workloads.

A common ML operation:

$$Y = XW$$

This matrix multiplication contains many independent calculations.

Connection

DNNs and Transformers use large numbers of matrix and tensor operations, which makes them well suited to GPU acceleration.

GPU Performance: Compute vs Memory

A GPU can be limited by:

- compute throughput,
- memory bandwidth,
- memory capacity,
- data transfer overhead.

$$\boxed{\text{Performance} = \min(\text{Compute Capability}, \text{Data Supply})}$$

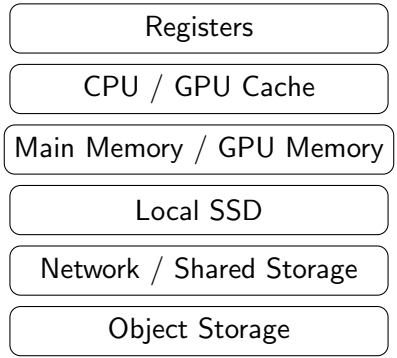
If the GPU is waiting for data, adding more compute units may not improve performance.

This connects directly to:

- storage formats,
- data loading,
- batch size,
- data pipelines,
- CPU preprocessing.

The Memory Hierarchy

Data can be stored at different levels:



Generally:

- closer memory is faster,
- closer memory is smaller,
- moving data can be expensive.

The Data Movement Problem

A training workload may require:

$$\text{Storage} \rightarrow \text{CPU} \rightarrow \text{GPU}$$

Each transfer can introduce latency.

If data cannot be supplied quickly enough: GPU utilisation $\downarrow$
Possible strategies:

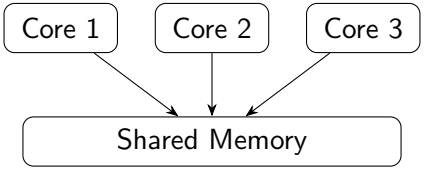
- prefetch data,
- use parallel data loaders,
- cache frequently used data,
- use efficient storage formats,
- reduce unnecessary transfers,
- use larger sequential reads where appropriate.

Connection to data storage lecture

Efficient storage and data pipelines are part of high-performance computing.

Shared-Memory Systems

In a shared-memory system, multiple CPU cores access the same main memory.



Advantages:

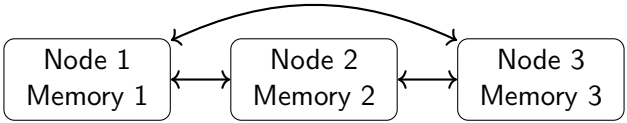
- relatively simple communication,
- shared access to data,
- useful for multi-threaded applications.

Limitations:

- memory bandwidth contention,
- synchronisation overhead,
- limited scalability within one machine.

Distributed-Memory Systems

In a distributed-memory system, each machine has its own memory.



Each node has its own:

- CPU,
- memory,
- operating system,
- local resources.

Nodes communicate over a network.

Why Distributed Computing Is Harder

Distributed systems introduce new costs:

- network latency,
- communication bandwidth,
- synchronisation,
- partial failures,
- data partitioning,
- consistency.

A local memory access is not equivalent to a network transfer.

Communication is part of the computation

A distributed algorithm must consider:

What is computed locally, and what must be communicated?

MPI: Message Passing Interface

MPI is a standard for communication between processes in distributed-memory systems.
Each process has its own memory. To exchange information, processes send/receive messages.

$$\text{Process } 0 \xrightarrow{\text{message}} \text{Process } 1$$

MPI supports:

- point-to-point communication,
- broadcast,
- scatter,
- gather,
- reductions,
- collective communication.

Connection

Distributed ML training needs communication to combine gradients, share parameters or exchange activations.

Collective Communication

Collective operations involve multiple processes. **Broadcast**
One process sends data to all processes.

$$P_0 \to P_1, P_2, \ldots, P_n$$

Reduce
Combine values from multiple processes.

$$x_0 + x_1 + \cdots + x_n$$

All-Reduce
Combine values and make the result available to all processes.

$$\boxed{\text{All-Reduce} = \text{Reduce} + \text{Distribute Result}}$$

This is particularly important for distributed SGD.

Distributed Data Parallel Training

Suppose we train a model using 4 GPUs.
Each GPU processes a different mini-batch:

$$B_1, B_2, B_3, B_4$$

Each computes a gradient:

$$g_1, g_2, g_3, g_4$$

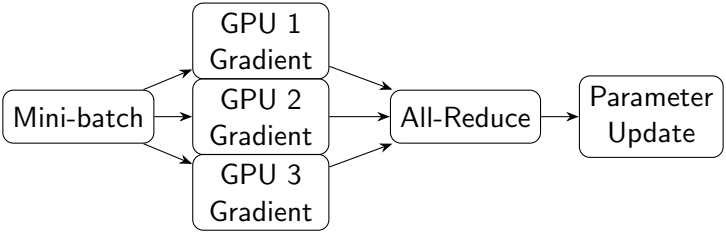
The gradients are combined:

$$g = \frac{1}{4}(g_1 + g_2 + g_3 + g_4)$$

Then each worker updates the model using the combined gradient.

Each GPU processes data in parallel, then synchronises.

Distributed Data Parallel: Workflow



This approach is commonly used when the model fits on each GPU but the training workload benefits from processing more data in parallel.

Scaling Distributed Training

More GPUs can increase throughput. But communication becomes important.
A simplified model:

$$T_{\text{step}} = T_{\text{compute}} + T_{\text{communication}} + T_{\text{overhead}}$$

If communication dominates:

$$T_{\text{communication}} \gg T_{\text{compute}}$$

then adding more GPUs may provide little benefit.
Performance depends on:

- model size,
- batch size,
- network bandwidth,
- communication pattern,
- hardware topology,
- implementation.

Model Parallelism

Sometimes a model is too large to fit on one GPU.
We can split the model across devices.
For example:

GPU 1 : Layers 1–4

GPU 2 : Layers 5–8

GPU 3 : Layers 9–12

This is a form of **model parallelism**.

Difference

Data parallelism splits the data.
Model parallelism splits the model computation or parameters.

Data Parallelism vs Model Parallelism

	Data Parallelism	Model Parallelism
Split	Training data	Model
Model copy	Usually on each worker	Partitioned across workers
Primary goal	Increase throughput	Fit / compute larger models
Communication	Gradient synchronisation	Activations / parameters
Common challenge	Synchronisation overhead	Inter-device communication

Large-scale training may combine:

$$\boxed{\text{Data Parallelism} + \text{Model Parallelism}}$$

HPC Workloads: Training vs Inference

Training:

- many forward and backward passes,
- gradient computation,
- parameter updates,
- often large batches,
- potentially long-running jobs.

Inference:

- forward pass only,
- latency-sensitive,
- may require many replicas,
- often driven by incoming requests.

$$\boxed{\text{Training} \neq \text{Inference}}$$

The hardware and deployment strategy may differ.

HPC and Hyperparameter Optimisation

Recall the hyperparameter search example. If we have 100 configurations:

$$\lambda_1, \lambda_2, \ldots, \lambda_{100}$$

Each configuration can potentially run as an independent job.
A cluster can execute multiple experiments concurrently.

$$\text{Experiment Scheduler} \rightarrow \begin{cases} \text{Training Job 1} \\ \text{Training Job 2} \\ \text{Training Job 3} \\ \vdots \end{cases}$$

Connection to MLOps

Experiment tracking and HPC complement each other: one manages the experiments, the other executes them at scale.

HPC and Job Scheduling

HPC resources are often shared by many users.
A scheduler decides:

- which jobs can run,
- when they can run,
- which nodes they use,
- how much CPU, memory and GPU they receive.

A job may request:

$$4 \text{ CPUs} + 32 \text{ GB RAM} + 1 \text{ GPU}$$

The scheduler places the job when appropriate resources are available.

This helps manage a shared cluster fairly and efficiently.

HPC Job Scheduling vs Kubernetes

Both HPC schedulers and Kubernetes manage compute resources, but they are often used for different workload patterns.

	HPC Scheduler	Kubernetes
Typical workload	Batch jobs	Services and container workloads
Common pattern	Submit / wait / run	Desired state / reconcile
Long computation	Common	Supported
Service scaling	Not primary focus	Core capability
Resource allocation	CPU / memory / GPU	CPU / memory / GPU

Examples of HPC schedulers include Slurm and PBS.
Kubernetes is commonly used for containerised services and can also run batch workloads.

Connecting HPC to Docker and OpenShift

$$\text{Docker} \rightarrow \text{Kubernetes} \rightarrow \text{OpenShift}$$

These technologies help package and manage workloads.
HPC adds a focus on:

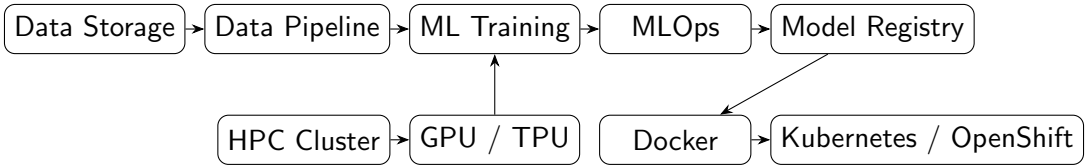
- parallel execution,
- high-throughput compute,
- large memory,
- fast interconnects,
- accelerator utilisation.

A container can package the application, while an HPC scheduler or Kubernetes-based platform allocates compute resources.

Important

Containers improve portability, but they do not automatically make an application parallel or high-performance.

A Unified ML Infrastructure View



Each layer addresses a different part of the system.

Performance Engineering: Where Is the Bottleneck?

Before adding more hardware, ask:

- Is the workload compute-bound?
- Is it memory-bound?
- Is it I/O-bound?
- Is it communication-bound?
- Is it limited by serial work?
- Is it waiting for a scheduler?

Measure → Identify Bottleneck → Optimise → Measure Again

Key principle

Do not assume that more hardware solves every performance problem.

Example: Training a Large Model

Suppose training takes 24 hours on one GPU.
Possible improvements:

1. Profile the training loop.
2. Check whether the GPU is fully utilised.
3. Optimise data loading.
4. Increase batch size if memory permits.
5. Use mixed precision where appropriate.
6. Use multiple GPUs with data parallelism.
7. Consider model parallelism if the model does not fit.
8. Track experiments and compare results.

The correct solution depends on the measured bottleneck.

HPC Challenges

HPC introduces additional engineering complexity:

- parallel algorithms are harder to design,
- communication can become expensive,
- failures may occur during long-running jobs,
- debugging is more difficult,
- resource allocation must be managed,
- performance can vary with workload and topology,
- reproducibility must be maintained.

Trade-off

More compute can improve performance, but it also introduces more complexity and operational cost.

Summary

- HPC enables computationally demanding workloads to run efficiently.
- Parallelism can be applied to independent tasks, data or models.
- CPUs, GPUs and other accelerators have different strengths.
- Data movement and communication are major performance costs.
- Shared-memory and distributed-memory systems have different trade-offs.
- MPI supports communication in distributed-memory systems.
- Distributed data parallel training combines gradients across workers.
- Model parallelism splits model computation across devices.
- HPC schedulers allocate resources for batch jobs.
- HPC complements MLflow, Docker, Kubernetes and OpenShift.

Discussion Questions

1. Why might 8 GPUs not be 8 times faster than one GPU?
2. What is the difference between data parallelism and model parallelism?
3. Why can a fast GPU be limited by storage?
4. What information does a scheduler need to allocate a job?
5. When might an HPC scheduler be a better fit than Kubernetes?
6. How could you parallelise hyperparameter optimisation?
7. What would you measure before scaling a training workload?